

Lecture 17

Cooperative Multitasking

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>



Cooperative Multitasking

- Even with a single thread*, we can perform several tasks “at once” if we switch back and forth between them.
- In **cooperative** multitasking, the key is that each task must relinquish control (pause, yield, await, etc.).
- There is no way for the runtime environment to “interrupt” a task/function that is currently still running code.

*Cooperative multitasking can certainly be used with multiple threads as well.



Version 0 – No Concurrency

```
function addAllToppings(pizza: Pizza) {  
  for (const topping of pizza.toppings) {  
    addTopping(pizza, topping);  
  }  
}  
  
function preparePizza(pizza: Pizza) {  
  const { color } = pizza;  
  prepareDough(pizza);  
  addAllToppings(pizza);  
  THE_FOUR_OVENS[0].bakeBlocking(pizza);  
  boxPizza(pizza);  
}
```

```
function main(pizza: Pizza) {  
  while (hasOrdersRemaining()) {  
    const order = checkPhone();  
    if (order) {  
      const pizza = createPizza(order);  
      preparePizza(pizza);  
    }  
  }  
}
```

Version 1 – Generators/Coroutines

```
function* addAllToppings(pizza: Pizza) {  
  for (const topping of pizza.toppings) {  
    addTopping(pizza, topping);  
    yield; // Yield after each topping  
  }  
}
```

```
function* bakeWithPolling(pizza: Pizza) {  
  // Wait for an available oven  
  let oven: Oven | undefined;  
  while (!(oven = findAvailableOven())) {  
    yield; // Yield and try again later  
  }  
  
  oven.startBaking(pizza);  
  while (!oven.checkIfDone()) {  
    yield; // Yield if still waiting  
  }  
}
```

```
function* preparePizza(pizza: Pizza) {  
  prepareDough(pizza);  
  yield;  
  yield* addAllToppings(pizza);  
  yield* bakeWithPolling(pizza);  
  boxPizza(pizza);  
}
```

```
function runScheduler() {  
  const task_queue: Coroutine[] = [];  
  while (task_queue.length > 0 || hasOrdersRemaining()) {  
    const order = checkPhone(); // Check phone every yield  
    if (order) {  
      task_queue.push(preparePizza(createPizza(order)));  
    }  
  
    // Run one step of one task  
    const nextTask = task_queue.shift();  
    if (nextTask) {  
      const result = nextTask.next();  
      if (!result.done) { task_queue.push(nextTask); }  
    }  
  }  
}
```

Version 2 - Callbacks

- This version is available in the lecture GitHub repository.

Version 3 - Promises

- This version is available in the lecture GitHub repository.

Version 4 - Async/Await

- This version is available in the lecture GitHub repository.